



CompTIA

DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

<https://www.DeepDumps.com>

About DeepDumps

At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.

With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://DeepDumps.com>

[Mail us on - support@deepdumps.com](mailto:support@deepdumps.com)



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John



Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Dana



Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You 🙏

Ahamod Shibly



Customer support is really fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Passed my exam today with 845 marks



Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

These questions are real and 100 % valid.



Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria



Simple, easy to understand explanations. To anyone who is writing the exam of AZ900,

Microsoft

(AI-103)

Developing AI Apps and Agents on Azure

Total: **135 Questions**

Link: <https://deepdumps.com/papers/microsoft/ai-103>

Question: 1

HOTSPOT -

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Ltd is a multinational retail company that builds, deploys, and manages generative AI and agent-based solutions by using Microsoft Foundry.

Existing Environment -

Identity Environment -

Contoso uses Microsoft Entra ID for identity management, authentication, and authorization capabilities that enable agents to access organizational resources and services.

Contoso recently formed a new AI engineering team named Agent1Dev Team to optimize and maintain existing AI solutions.

The team collaborates with solution architects, DevOps engineers, and security engineers to design, implement, monitor, and secure AI applications.

Contoso also has a team named Agent1Test Team that is responsible for validating AI solutions before the solution deployments.

Generative Environment -

Contoso has a Microsoft Foundry deployment that contains two projects named Project1 and Project2.

Project1 -

Project1 contains a customer support agent named Agent1 that assists customers with product inquiries and troubleshooting requests.

Agent1 has the following configurations:

Agent1 uses a base model deployment.

A safety evaluation pipeline is NOT enabled.

Tool invocation approval workflows are NOT enabled.

Conversation memory constraints are NOT configured.

Agent1 interacts with customers by using digital support channels and answers general questions about Contoso products.

Project1 is deployed to an Azure region located in the European Union (EU).

Agent1Dev Team will use Project1 to optimize and maintain Agent1.

Project2 -

Project2 contains a deployed video generation model. The marketing department at Contoso has access to Project2 and plans to use the model to develop a video creation solution.

Development of the solution is incomplete.

Data Environment -

Contoso stores product-related information in Azure resources that support AI applications.

The Azure environment contains an Azure Blob Storage account named storage1 that stores product detail sheets for all the Contoso products.

The product sheets include specifications, feature descriptions, and product support information that Agent1 can use to answer customer questions. The product sheets are stored in the PDF format.

Problem Statements -

Contoso identifies the following issues:

Agent1 has only general knowledge of the Contoso products.

A recent chat interaction with Agent1 was analyzed for sentiment. The results of the analysis have NOT been processed yet.

Agent1 does NOT use the detailed product information in the product sheets stored in storage1 when responding to customer questions.

The finance department at Contoso reports that vendor invoices must be reviewed manually to ensure that the invoices match the terms defined in the vendor contracts. The invoices contain tables, logos, and varied layouts that make the documents difficult to process consistently.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement a solution for Project1 that analyzes the vendor invoices by evaluating both the visual layout and the textual content of the invoices, so that the invoice details can be verified against the vendor contract terms.

Update the base model deployment used by Agent1 and standardize the model version to ensure continuity and consistent responses.

Enable Agent1 to retrieve and use the detailed product information from the product sheets stored in storage1.

Implement an indexing solution for the product sheets that Agent1 can use to answer customer questions.

Complete the development of the video creation solution.

Technical Requirements -

Contoso identifies the following technical requirements:

The model deployment used by Agent1 must support scalable, high-throughput generative AI workloads and dynamically scale to handle variable customer support traffic, without requiring reserved throughput capacity.

The product sheets must be processed by using an indexing pipeline that enables semantic and vector search, so that Agent1 can retrieve the relevant product information.

Responses generated by using the product sheet information must be relevant, complete, and accurate.

Agent1 must be able to use the product sheets to answer natural language questions about product details.

The model version used by Agent1 must remain consistent to ensure stable responses.

The data processed by the model must remain within the EU.

Security and Compliance Requirements

Contoso identifies the following security and compliance requirements:

API keys must NOT be used to access Foundry-deployed models.

Access to the Azure resources must follow the principle of least privilege.

The developers at Contoso must authenticate to Microsoft Foundry resources by using Microsoft Entra authentication.

Access to Project1 must be assigned to the members of Agent1Dev Team by using a security group named SC_Agent1_Dev.

Access to Project1 must be assigned to the members of Agent1Test Team by using a security group named SC_Agent1_Test.

Agent1 must never reveal customer information, even if a document that contains customer data is added erroneously to the product sheet repository in storage1.

The product sheets might contain images that include embedded text. Agent1 must be protected from malicious instructions potentially hidden within the images.

Business Requirements -

Contoso identifies the following business requirements:

Users that interact with Agent1 must have a personalized experience in future interactions, including the ability for Agent1 to retain conversation context and recall relevant information from previous interactions.

Agent1 must answer questions only about the products sold by Contoso.

You need to configure the model deployment for Agent1 to meet the technical requirements.

What should you configure? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Deployment type:

Standard
Global Standard
Global Provisioned

Version update policy:

Once the current version expires
Opt out of automatic model version upgrades
Upgrade once a new default version becomes available

Answer:

Answer Area

Deployment type:

- Standard
- Global Standard
- Global Provisioned

Version update policy:

- Once the current version expires
- Opt out of automatic model version upgrades
- Upgrade once a new default version becomes available

Explanation:

Deployment Type: Selected as **Standard** because the workload requires dynamic scaling for variable traffic without reserved throughput capacity (eliminating Provisioned options), while adhering to strict regional data residency guidelines (e.g., data remaining within a specific zone or region like the EU).

Version Update Policy: Selected as **Once the current version expires** because the model version must remain consistent to guarantee stable, predictable responses over time, rather than automatically upgrading as soon as a new default version becomes available.

Why the other answer are incorrect:

Global Standard: Incorrect because "Global" routing sends traffic dynamically to any region worldwide with available capacity. This violates strict **data residency/compliance** rules if your data must remain within a specific geographic boundary (like the EU or US).

Global Provisioned: Incorrect because "Provisioned" requires buying reserved Throughput Units (PTUs). This incurs a high, fixed cost regardless of usage, making it wrong for workloads with **variable, low-volume traffic** where minimizing costs is a priority.

Opt out of automatic model version upgrades: Incorrect because this policy is deprecated or not recommended for long-term consistency. When a model version reaches its official retirement date, it will force-upgrade anyway, meaning you cannot permanently opt out of upgrades.

Upgrade once a new default version becomes available: Incorrect because this causes the model to update automatically as soon as Microsoft releases a new default. This can **break application code** or change prompt behaviors unexpectedly, violating requirements for strict consistency.

Question: 2

Deep Dumps

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to

return to it.

To start the case study -

To display the first question in this case study, select the “Next” button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the “Question” button to return to the question.

Overview -

Company Information -

Contoso, Ltd is a multinational retail company that builds, deploys, and manages generative AI and agent-based solutions by using Microsoft Foundry.

Existing Environment -

Identity Environment -

Contoso uses Microsoft Entra ID for identity management, authentication, and authorization capabilities that enable agents to access organizational resources and services.

Contoso recently formed a new AI engineering team named Agent1Dev Team to optimize and maintain existing AI solutions.

The team collaborates with solution architects, DevOps engineers, and security engineers to design, implement, monitor, and secure AI applications.

Contoso also has a team named Agent1Test Team that is responsible for validating AI solutions before the solution deployments.

Generative Environment -

Contoso has a Microsoft Foundry deployment that contains two projects named Project1 and Project2.

Project1 -

Project1 contains a customer support agent named Agent1 that assists customers with product inquiries and troubleshooting requests.

Agent1 has the following configurations:

Agent1 uses a base model deployment.

A safety evaluation pipeline is NOT enabled.

Tool invocation approval workflows are NOT enabled.

Conversation memory constraints are NOT configured.

Agent1 interacts with customers by using digital support channels and answers general questions about Contoso products.

Project1 is deployed to an Azure region located in the European Union (EU).

Agent1Dev Team will use Project1 to optimize and maintain Agent1.

Project2 -

Project2 contains a deployed video generation model. The marketing department at Contoso has access to Project2 and plans to use the model to develop a video creation solution.

Development of the solution is incomplete.

Data Environment -

Contoso stores product-related information in Azure resources that support AI applications.

The Azure environment contains an Azure Blob Storage account named storage1 that stores product detail sheets for all the Contoso products.

The product sheets include specifications, feature descriptions, and product support information that Agent1 can use to answer customer questions. The product sheets are stored in the PDF format.

Problem Statements -

Contoso identifies the following issues:

Agent1 has only general knowledge of the Contoso products.

A recent chat interaction with Agent1 was analyzed for sentiment. The results of the analysis have NOT been processed yet.

Agent1 does NOT use the detailed product information in the product sheets stored in storage1 when responding to customer questions.

The finance department at Contoso reports that vendor invoices must be reviewed manually to ensure that the invoices match the terms defined in the vendor contracts. The invoices contain tables, logos, and varied layouts that make the documents difficult to process consistently.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement a solution for Project1 that analyzes the vendor invoices by evaluating both the visual layout and the textual content of the invoices, so that the invoice details can be verified against the vendor contract terms.

Update the base model deployment used by Agent1 and standardize the model version to ensure continuity and consistent responses.

Enable Agent1 to retrieve and use the detailed product information from the product sheets stored in storage1.

Implement an indexing solution for the product sheets that Agent1 can use to answer customer questions.

Complete the development of the video creation solution.

Technical Requirements -

Contoso identifies the following technical requirements:

The model deployment used by Agent1 must support scalable, high-throughput generative AI workloads and dynamically scale to handle variable customer support traffic, without requiring reserved throughput capacity. The product sheets must be processed by using an indexing pipeline that enables semantic and vector search, so that Agent1 can retrieve the relevant product information.

Responses generated by using the product sheet information must be relevant, complete, and accurate. Agent1 must be able to use the product sheets to answer natural language questions about product details. The model version used by Agent1 must remain consistent to ensure stable responses.

The data processed by the model must remain within the EU.

Security and Compliance Requirements

Contoso identifies the following security and compliance requirements:

- API keys must NOT be used to access Foundry-deployed models.
- Access to the Azure resources must follow the principle of least privilege.
- The developers at Contoso must authenticate to Microsoft Foundry resources by using Microsoft Entra authentication.
- Access to Project1 must be assigned to the members of Agent1Dev Team by using a security group named SC_Agent1_Dev.
- Access to Project1 must be assigned to the members of Agent1Test Team by using a security group named SC_Agent1_Test.
- Agent1 must never reveal customer information, even if a document that contains customer data is added erroneously to the product sheet repository in storage1.
- The product sheets might contain images that include embedded text. Agent1 must be protected from malicious instructions potentially hidden within the images.

Business Requirements -

Contoso identifies the following business requirements:

- Users that interact with Agent1 must have a personalized experience in future interactions, including the ability for Agent1 to retain conversation context and recall relevant information from previous interactions.
- Agent1 must answer questions only about the products sold by Contoso.
- You need to configure Agent1 to meet the security and compliance requirements.

What should you use?

- A. self-harm content filtering
- B. prompt shields
- C. Personally identifiable information (PII) Detection
- D. violence content filtering

Answer: B

Explanation:

Technical Justification for Correct Answer: B. Prompt Shields

Why B (Prompt Shields) is the best option:

Security and Compliance Requirement Alignment: The primary reason for choosing Prompt Shields is its direct alignment with multiple security and compliance requirements stated by Contoso. Specifically, it addresses the need to protect Agent1 from revealing customer information (even when erroneously exposed) and from potential malicious instructions hidden within images in product sheets.

Protection Against Malicious Inputs: Prompt Shields are designed to filter out or shield against potentially harmful or unwanted inputs (including those that might be embedded in images), ensuring Agent1's responses remain safe and compliant.

Relevance to General Security Posture: Given the constraints around not using API keys and the emphasis on least privilege access, leveraging Prompt Shields further fortifies the security posture of Agent1 without introducing additional access vulnerabilities.

Why Other Options are Less Suitable:

A. Self-Harm Content Filtering:

Relevance: While important, self-harm content filtering does not directly address the specified security and compliance requirements related to protecting against malicious instructions or inadvertently exposed

customer data.

Scope: The primary concern here is not the content generated by Agent1 in response to self-harm queries but ensuring the security of the input process.

C. Personally Identifiable Information (PII) Detection:

Relevance: Although PII Detection is crucial for data privacy, the question's focus is on protecting Agent1 from malicious inputs and ensuring it doesn't reveal customer information **proactively**. PII Detection is more about identifying than preventing exposure.

Proactivity: The requirement implies a need for a proactive protection mechanism rather than a detection capability.

D. Violence Content Filtering:

Relevance: Similar to self-harm content filtering, this does not directly address the protection against malicious instructions or the accidental exposure of customer information as highlighted in the problem statement.

Scope: The concern is broader than just filtering violent content; it's about securing the input pipeline.

Conclusion: Given the specific security and compliance requirements outlined by Contoso, particularly the need to protect against malicious inputs and prevent the exposure of customer information, **Prompt Shields (B)** is the most appropriate choice. It directly addresses the identified risks without introducing additional vulnerabilities, aligning with the principle of least privilege and enhancing the overall security posture of Agent1.

References

[Microsoft Documentation: security and compliance for Azure AI Services](#)

[Microsoft Learn: Protecting AI Models from Malicious Inputs](#)

Question: 3

Deep Dumps

You are planning a Microsoft Foundry project named Project1 that will contain multiple agents. Each agent will access the same Azure AI Search resource.

You need to recommend a solution to centrally manage the Azure AI Search credentials within Project1. The solution must be implemented across all the agents.

What should you recommend?

- A. Enable role-based access control (RBAC) for the Azure AI Search resource.
- B. Disable key-based access control on the Azure AI Search resource.
- C. Add a connection to the Azure AI Search resource.
- D. Create a managed private endpoint that connects to the Azure AI Search resource.

Answer: C

Explanation:

Technical Justification for Recommending Option C

For centrally managing Azure AI Search credentials across multiple agents in Project1, **Option C: Add a connection to the Azure AI Search resource** is the most suitable choice. Here's why:

Option C is the best because adding a connection to the Azure AI Search resource within the Microsoft Foundry project (Project1) allows for a centralized management approach. This connection can be securely shared or referenced across all agents within the project, ensuring that credentials are not hardcoded or duplicated across agents. Instead, agents can leverage this single, managed connection, enhancing security and simplifying credential updates.

Why Other Options are Less Suitable:

Option A: Enable role-based access control (RBAC) for the Azure AI Search resource

While RBAC is crucial for controlling access, it does not directly address the central management of credentials for accessing the Azure AI Search resource. RBAC defines what actions can be performed but does not consolidate credential management across multiple agents.

Suitability for Central Credential Management: Low

Option B: Disable key-based access control on the Azure AI Search resource

Disabling key-based access would likely increase security risks by potentially forcing the use of less secure methods or complicating access management. This does not contribute to centralizing credential management.

Suitability for Central Credential Management: Very Low (Counterproductive)

Option D: Create a managed private endpoint that connects to the Azure AI Search resource

While a managed private endpoint enhances security by providing a private, secure connection to the Azure AI Search resource, it focuses on network security rather than the central management of credentials for access by multiple agents.

Suitability for Central Credential Management: Low

Recommendation Summary: Given the need for central management of Azure AI Search credentials across multiple agents in Project1, **Option C** is the most direct and effective solution, facilitating secure, centralized credential management.

References:

For further understanding of managing connections and security in Azure and Microsoft Foundry, refer to:

1. **Microsoft Documentation - Manage connections in Microsoft Foundry:**
<https://learn.microsoft.com/en-us/microsoft-365/apps/foundry/manage-connections?view=o365-worldwide>
2. **Azure Documentation - Secure your Azure AI Search with Azure RBAC:**
<https://learn.microsoft.com/en-us/azure/search/security-azure-rbac>

Question: 4

Deep Dumps

HOTSPOT -

Your company is piloting a customer support agent in a Microsoft Foundry project name Project1. Project1 is connected to an existing Application Insights resource, and the company's support team reviews runs in the Traces tab.

The Foundry Agent Service is configured to perform the following actions:

Retrieve the Application Insights connection string by calling

`project_client.telemetry.get_application_insights_connection_string()`.

Call `configure_azure_monitor(connection_string=...)` to enable telemetry.

A separate LangChain service is configured to use OpenTelemetry and has the following configurations:

Uses `AzureAIOpenTelemetryTracer(connection_string=..., enable_content_recording=False)`

Passes the tracer by using `config= "callbacks":[azure_tracer]`

Company policy has the following requirements:

Telemetry from LangChain and OpenTelemetry must be distinguishable within the same Application Insights resource.

Secrets and credentials must NOT be stored in prompts, tool arguments, or span attributes.

For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The LangChain service will appear in Traces without configuring a tracer.	☐	☐
Setting different OTEL_SERVICE_NAME values separates the services in Application Insights.	☐	☐
When using enable_content_recording=False, prompts and tool data will be captured in the telemetry.	☐	☐

Answer:

Answer Area

Statements	Yes	No
The LangChain service will appear in Traces without configuring a tracer.	☐	☒
Setting different OTEL_SERVICE_NAME values separates the services in Application Insights.	☒	☐
When using enable_content_recording=False, prompts and tool data will be captured in the telemetry.	☐	☒

Explanation:

Statement 1: No

OpenTelemetry tracing requires explicit configuration to start capturing data. Without setting up an active OpenTelemetry exporter or attaching a dedicated tracer (such as AzureAIOpenTelemetryTracer or calling `configure_azure_monitor`), no trace spans or events will be captured or forwarded to Azure Monitor.

Statement 2: Yes

The OTEL_SERVICE_NAME environment variable determines the logical name of your application service in OpenTelemetry telemetry data. Configuring distinct values for different services allows [Azure Application Insights](#) to map, filter, and isolate them into distinct components within the Application Map view.

Statement 3: No

Setting `enable_content_recording=False` explicitly tells the tracer to **exclude** the actual text content of prompts, completions, and tool data payloads from the telemetry attributes. This protects user privacy and prevents sensitive personal data or secrets from being stored inside logs.

Question: 5

Deep Dumps

DRAG DROP -

You have a Microsoft Foundry project that processes procurement documents submitted by suppliers. You need to implement two pipelines by using Azure Content Understanding in Foundry Tools. The solution must meet the following requirements:

Include a pipeline named Pipeline1 that supports cost-effective, high-volume processing of standalone PDF

invoices.
Include a pipeline named Pipeline2 that supports cross-document validation by using multi-step reasoning and reference data.
How should you configure each pipeline? To answer, drag the appropriate configurations to the correct pipelines. Each configuration may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Configurations

Multi-file task in pro mode

Multi-file task in standard mode

Single-file task in pro mode

Single-file task in standard mode

Answer Area

Pipeline1: Configuration

Pipeline2: Configuration

Answer:

Configurations

Multi-file task in pro mode

Multi-file task in standard mode

Single-file task in pro mode

Single-file task in standard mode

Answer Area

Pipeline1: Single-file task in standard mode

Pipeline2: Multi-file task in pro mode

Explanation:

Pipeline1 (Single-file task in standard mode):

This is correct for workflows processing independent, standalone documents one by one. [Standard mode](#) handles individual file parsing efficiently. It minimizes computational costs for basic schema extraction.

Pipeline2 (Multi-file task in pro mode):

This is correct for workflows requiring cross-document logic or advanced generative reasoning. Pro mode processes batches of interrelated files together. It allows the model to correlate information across multiple files.

Incorrect answer:

Single-file task in pro mode:

This is incorrect because Pro mode's core advantage is cross-file reasoning. Using it for a single file unnecessarily increases processing limits and costs without utilizing multi-file capabilities.

Multi-file task in standard mode:

This is incorrect because Standard mode lacks the architecture to handle relational dependencies across multiple files. Combining multiple files under standard mode results in schema rejection or validation errors.

Question: 6

Deep Dumps

HOTSPOT -

You have a Python application named App1 that integrates with a Microsoft Foundry project named Project1. You need to ensure that App1 meets the following requirements:

Authenticates by using a Microsoft Entra managed identity

Sends prompts to a deployed model by using the Azure OpenAI Responses API

How should you complete the Python code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
from azure.identity import DefaultAzureCredential

from azure.ai.projects import AIProjectClient

credential =  ()

project_client = AIProjectClient(
    endpoint="https://contosoai.services.ai.azure.com/api/projects/project1",
    credential=credential,
)

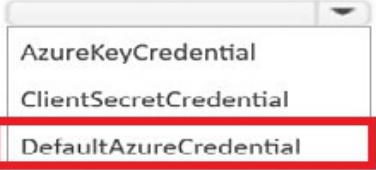
with project_client.get_openai_client() as openai_client:
    response = openai_client.responses.  (

        model="trail-guide-chat",
        input="Create a 3-day hiking itinerary near Seattle.",
    )

print(response.output_text)
```

Answer:

Answer Area

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
credential =  ()

project_client = AIProjectClient(
    endpoint="https://contosoai.services.ai.azure.com/api/projects/project1",
    credential=credential,
)

with project_client.get_openai_client() as openai_client:
    response = openai_client.responses.  (

        model="trail-guide-chat",
        input="Create a 3-day hiking itinerary near Seattle.",
    )

print(response.output_text)
```

Explanation:

First Dropdown: DefaultAzureCredential

Second Dropdown: create

Credential Initialization: The scenario specifies authentication using a **Microsoft Entra managed identity**. In Azure Python SDKs, [DefaultAzureCredential](#) automatically orchestrates managed identities across both local development and Azure-hosted environments. It is explicitly imported at the top of the snippet (from `azure.identity import DefaultAzureCredential`).

Responses API Call: When interacting with the Azure OpenAI-compatible project client endpoint via `openai_client.responses`, the standard method to generate chat text completions or send user prompts to the deployed generative model is [create](#).

Question: 7

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains a workflow for a customer support triage process.

You have an Ask a question node that stores user responses in a local variable named Var01.

You need to create the following Power Fx expressions:

An if/else condition expression that ensures that Var01 contains a value

A Send message expression that returns the stored user response in uppercase

How should you configure the expressions? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

If/else condition expression:

```
IsBlank(Local.Var01)
IsEmpty(Local.Var01)
Not(IsBlank(Local.Var01))
```

Send message expression:

```
{Local.Var01}
{Upper(Local.Var01)}
{Upper(Var01)}
```

Answer:

Answer Area

If/else condition expression:

```
IsBlank(Local.Var01)
IsEmpty(Local.Var01)
Not(IsBlank(Local.Var01))
```

Send message expression:

```
{Local.Var01}
{Upper(Local.Var01)}
{Upper(Var01)}
```

Explanation:

If/else condition expression: **Not(IsBlank(Local.Var01))**

Send message expression: **Upper(Local.Var01)**

If/else condition expression:

The requirement is to ensure that the variable **contains a value** (i.e., it is not empty or blank).

IsBlank(Local.Var01) evaluates to True if it has no value.

Wrapping it in the Not() function inverts this logic, making the expression evaluate to True only when a valid value exists.

Send message expression:

To display or send a variable dynamically inside a message node, the variable/formula must be enclosed within curly brackets string interpolation syntax.

The Upper() function converts the text string to uppercase characters.

The variable prefix Local. must be explicitly retained to reference the locally-scoped variable correctly within the expression payload (Upper(Local.Var01)).

Question: 8

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains a customer support agent built by using the Foundry Agent Service.

The agent uploads user-provided screenshots to Azure Storage through a ticketing tool and receives a blob URL for additional reasoning.

You need to use image moderation during agent runs and prevent harmful content from being returned during runs. Azure AI Content Safety must access the images by using the blob URL. The solution must follow the principle of least privilege.

What should you configure for Content Safety? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Guardrails:

- Select Tool call and set Action to Block.
- Select User input and Output and set Action to Annotate.
- Select User input and Tool response and set Action to Annotate.
- Select User input, Output, Tool response, and Tool call and set Action to Block.

Storage access:

- Storage account access keys
- A user-assigned identity that is assigned the Storage Queue Data Contributor role
- A system-assigned managed identity that is assigned the Storage Blob Data Reader role
- A system-assigned managed identity that is assigned the Storage Blob Data Contributor role

Answer:

Answer Area

Guardrails:

- Select Tool call and set Action to Block.
- Select User input and Output and set Action to Annotate.
- Select User input and Tool response and set Action to Annotate.
- Select User input, Output, Tool response, and Tool call and set Action to Block.

Storage access:

- Storage account access keys
- A user-assigned identity that is assigned the Storage Queue Data Contributor role
- A system-assigned managed identity that is assigned the Storage Blob Data Reader role
- A system-assigned managed identity that is assigned the Storage Blob Data Contributor role

Explanation:

Guardrails

Select User input, Output, Tool response, and Tool call and set Action to Block.

Storage access

A system-assigned managed identity that is assigned the Storage Blob Data Reader role

To securely evaluate and prevent harmful or unsafe content from being returned or processed at any point during an agentic session, content safety filters must be applied comprehensively across all interaction touchpoints. Setting the action to **Block** ensures that any policy violations are immediately halted rather than merely flagged or annotated in logs, completely protecting the user interface from receiving or exposing toxic data.

Storage access:

When Azure AI Content Safety retrieves screenshots or file attachments via direct blob URLs, it requires specific data-plane permissions on the underlying storage repository.

System-assigned managed identity eliminates password management overhead.

Storage Blob Data Reader satisfies the principle of **least privilege**, providing full read access to download and review image binaries without granting unnecessary write or delete capabilities (which would be included in the Contributor role).

Question: 9

Deep Dumps

You have a Microsoft Foundry project that contains three agents as shown in the following table.

Name	Description
TriageAgent	Classifies incoming customer requests
PolicyAgent	Answers policy questions by searching internal content
ActionAgent	Creates or updates tickets by calling an HTTP API

You need to orchestrate the agents to ensure that the customer requests meet the following requirements: Support a deterministic, step-based process that uses conditional branching and shared state across the agents. Optionally trigger a ticket action based on the triage result. The solution must minimize development effort. What should you include in the solution?

- A. a workflow
- B. threads and runs without a workflow
- C. a multi-agent group chat session
- D. separate agent runs coordinated in the application code

Answer: A

Explanation:

A. a workflow.

Why a workflow is correct: In Microsoft Azure AI Foundry, agentic workflows are purposely built to orchestrate multiple agents using declarative, predefined sequences. They natively support deterministic step-by-step logic, if/else conditional branching, and automatic variable/state sharing across agents without requiring developers to write complex application orchestration or synchronization code, minimizing overall development effort.

Why other options are incorrect:

Group chat sessions (C) are designed for autonomous, fluid multi-agent conversations where agents dynamically choose when to chime in or pass control. They do not natively follow strict, deterministic, step-based workflows.

Separate application code loops or standalone threads (B & D) require significant manual engineering effort to establish shared states, handle condition mappings, and write error-handling glue logic.

Question: 10**Deep Dumps**

You have a Microsoft Foundry project that contains an agent. The agent uses Azure Speech in Foundry Tools. You fine-tune a baseline speech to text model for the en-us locale and publish the model. The agent calls the Speech to text REST API and returns an error message indicating that the project ID is invalid. You need to set the project property to the correct ID. To what should you set the project property?

- A. the project URL
- B. the custom speech project ID
- C. the project ID
- D. the custom speech endpoint URL

Answer: B**Explanation:****Technical Justification for Correct Answer: B**

The correct option to set the project property to resolve the invalid project ID error when using Azure Speech in a Microsoft Foundry project is **B. the custom speech project ID**. Here's why:

Why B is Correct: When fine-tuning a baseline speech-to-text model for a specific locale (like en-us) and publishing it through Azure Speech Services, a unique **Custom Speech Project ID** is generated. This ID is crucial for identifying your specific, customized model when making API calls. The Speech to Text REST API requires this exact ID to route the request to your customized model, ensuring the correct speech-to-text processing. Setting the project property to the **custom speech project ID** ensures that the API call targets your fine-tuned model, resolving the invalid project ID error.

Why Other Options are Less Suitable:

A. the project URL: While the project URL might contain the project ID as part of its path, using the full URL is not the expected format for the project ID field in the API configuration. The API typically expects just the ID, not the entire URL, to identify the project.

C. the project ID: This option is misleadingly similar to the correct answer. In the context of Azure Speech Services, "project ID" could ambiguously refer to a higher-level project container ID (not specific to the customized speech model). The **custom speech project ID** (Option B) is more precise, indicating it's the ID for the fine-tuned speech model, not just any project ID.

D. the custom speech endpoint URL: Similar to Option A, using the full endpoint URL is not appropriate for the project ID field. The endpoint URL is used to direct the API call to the speech service's entry point, but the project ID (specifically the custom speech project ID for your model) is still required within the API call parameters to identify which model to use.

Technical Summary: The error indicates a mismatch in identifying your customized speech-to-text model. By setting the project property to the **custom speech project ID**, you ensure the API call accurately references your published, fine-tuned model for the en-us locale, thereby resolving the invalid project ID error.

References

[Azure Speech Services Documentation - Custom Speech](#)

[Azure Speech Services REST API Reference - Speech to Text](#)

Question: 11**Deep Dumps**

HOTSPOT -

You have a Microsoft Foundry project that contains an agent named PaymentAgent.

PaymentAgent includes a function tool that issues customer refunds by using an external API.

You are creating a workflow in YAML.

You need to ensure that the workflow pauses for human approval and continues with the refund step only after approval is granted.

How should you complete the workflow definition? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
steps:
```

```
- id: propose_refund
```

```
  type: agent
```

```
  agent: PaymentAgent
```

```
- id: approval
```

```
  type:
```

ask_question

basic_chat

data_transformation

```
- id: execute_refund
```

```
  type: agent
```

```
  agent: PaymentAgent
```

```
  condition:
```

approval == "approved"

propose_refund.output != null

true

Answer:

Answer Area

steps:

- id: propose_refund

type: agent

agent: PaymentAgent

- id: approval

type:

ask_question

basic_chat

data_transformation

- id: execute_refund

type: agent

agent: PaymentAgent

condition:

approval == "approved"

propose_refund.output != null

true

Explanation:

ask_question: In declarative agent workflows, ask_question is used to explicitly pause execution and collect external feedback or manual intervention (such as human authorization/approval) before proceeding, unlike basic_chat or data_transformation which run automatically.

approval == "approved" To ensure that the execute_refund step runs conditionally only if approval has been explicitly granted, the expression evaluates the string output from the preceding id: approval step against the expected value "approved".

Question: 12

Deep Dumps

You have an Azure Speech in Foundry Tools resource that hosts a custom speech to text model deployed to a custom endpoint. An agent uses the endpoint to perform real-time speech recognition. You are approaching the expiration date of the custom speech to text model. What is the expected behavior when the model expires?

- A. Speech recognition requests will return a 4xx error until a new custom model is deployed.
- B. Speech recognition requests will continue to use the expired custom model until the model is removed manually.
- C. Speech recognition requests will fall back to the most recent base model for the same locale.

D. The custom model will be deleted automatically when the model expires.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct answer is **C. Speech recognition requests will fall back to the most recent base model for the same locale**. Here's why:

Reason for C being correct: When a custom speech-to-text model expires in Azure Speech Services (formerly part of Cognitive Services, now potentially managed through Azure Speech in Foundry Tools as mentioned), the service is designed to ensure continuity of service. Instead of interrupting the functionality completely (which would be the case with a 4xx error as suggested by A) or requiring immediate manual intervention, Azure Speech Services automatically falls back to the **most recent base model** for the **same locale** as the expired custom model. This fallback mechanism minimizes disruption to the agent's speech recognition capabilities, ensuring some level of functionality is maintained until a new custom model is deployed.

Why A is less suitable:

A. Speech recognition requests will return a 4xx error until a new custom model is deployed. This option suggests a complete service interruption upon model expiration, which contradicts the design principle of Azure Services to provide resilient and continuous functionality wherever possible. A 4xx error implies a client-side issue or a deliberate block, which doesn't align with the expected behavior for a managed expiration scenario.

Why B is less suitable:

B. Speech recognition requests will continue to use the expired custom model until the model is removed manually. Continuing to use an expired model could pose security, compliance, or accuracy issues, as the model's support and potential updates would have ceased. Azure services typically do not extend the use of expired resources in such a manner without explicit configuration for a temporary extension or auto-renewal, which is not implied here.

Why D is less suitable:

D. The custom model will be deleted automatically when the model expires. While automatic deletion might seem like a housekeeping measure, the immediate question concerns the behavior of **speech recognition requests** at the time of expiration, not the model's storage lifecycle. The automatic model deletion (if it were to happen immediately, which is not the primary concern here) does not directly address the continuity of the speech recognition service's functionality.

Correct Answer Summary

Correct Answer: C. Speech recognition requests will fall back to the most recent base model for the same locale.

Rationale: Ensures service continuity with the best available alternative (latest base model for the locale) upon custom model expiration, aligning with Azure's design for resilient services.

References

[Azure Speech Services: Custom Speech Models](#)

[Azure Speech Services: How to manage models](#)

Question: 13

Deep Dumps

You have a Microsoft Foundry project that contains a model deployment. You have an application that calls the deployment by using the Azure OpenAI v1 API and DefaultAzureCredential. The developers at your company receive HTTP 403 errors when they send inference requests, even after running az login.

You need to ensure that the developers can perform model inference. The solution must follow the principle of least privilege.

Which role-based access control (RBAC) role should you assign to the developers?

- A. Cognitive Services User
- B. Cognitive Services OpenAI User
- C. Contributor
- D. Cognitive Services Data Reader

Answer: B

Explanation:

Technical Justification for Correct Answer: B (Cognitive Services OpenAI User)

The correct role-based access control (RBAC) role to assign to the developers for resolving the HTTP 403 errors when sending inference requests to a Microsoft Foundry (formerly Azure Machine Learning) model deployment via the Azure OpenAI v1 API, while adhering to the principle of least privilege, is **B. Cognitive Services OpenAI User**. Here's why:

Cognitive Services OpenAI User (B): This role is specifically designed for users who need to perform OpenAI model inference. Assigning this role ensures that developers have the necessary permissions to call the OpenAI API for model inference without granting excessive privileges. This aligns with the principle of least privilege, as it only allows actions directly related to the task at hand (OpenAI model inference) and nothing more.

Why Other Options are Less Suitable:

A. Cognitive Services User: While this role provides access to Cognitive Services, it is more general and might not specifically grant the necessary permissions for OpenAI model inference, or it could grant broader access than necessary, violating the principle of least privilege.

C. Contributor: Assigning the Contributor role would grant far too many permissions, including the ability to manage resources, which is not necessary for simply performing model inference. This vastly exceeds the principle of least privilege.

D. Cognitive Services Data Reader: This role is focused on reading data from Cognitive Services resources but does not necessarily provide the execution permissions required for model inference via the OpenAI API, making it inappropriate for this specific task.

Principle of Least Privilege Alignment: Assigning **Cognitive Services OpenAI User** ensures that developers have only the permissions necessary to perform their tasks (model inference via OpenAI API), without any additional, potentially risky permissions.

References

1. **Azure RBAC for Cognitive Services and OpenAI:** <https://learn.microsoft.com/en-us/azure/cognitive-services/authentication-acls?tabs=platform>
2. **Azure OpenAI Permissions and Roles:** <https://learn.microsoft.com/en-us/azure/cognitive-services/openai/how-to-use-manager-keys#permissions-and-roles>

Question: 14

Deep Dumps

You have a Microsoft Foundry project that contains an agent. The agent has a Model Context Protocol (MCP) tool that queries a knowledge base stored in Azure AI Search.

Some agent runs return answers from the base model without invoking the knowledge base, which results in responses without grounded citations.

You are provided with the following code snippet that runs the agent.

```
run = project_client.agents.runs.create_and_process(  
    thread_id=thread.id,  
    agent_id=agent.id  
)
```

You need to add the correct tool_choice parameter to the code to deterministically force the agent to invoke the MCP tool on each run.

What should you add?

- A. tool_choice= "required"
- B. tool_choice= "auto"
- C. tool_choice= "type": "knowledge_base"
- D. tool_choice = "type": "mcp"

Answer: A

Explanation:

A. tool_choice= "required" .

Why it is correct: According to [Microsoft Foundry Agent Service tool best practices](#), the tool_choice parameter provides runtime control over whether a model is allowed to answer directly or is forced to invoke a tool. Setting tool_choice="required" (or passing the parameter mode as a dict/object depending on the specific wrapper structure) explicitly tells the orchestrator that the agent **must execute one or more tools** before formulating its final response. Since the agent only has the Model Context Protocol (MCP) tool configured for querying the knowledge base, this forces deterministic tool execution on every single run, preventing ungrounded base model hallucinations.

Why the other options are incorrect:

B ("auto"): This is the default setting. It lets the model autonomously decide whether to use a tool, which makes it non-deterministic and leads to runs skipping the knowledge base entirely.

C & D: These options represent invalid formatting syntax for directing type scopes or target keywords within the standard parameter layout for tool-constrained invocation under the Foundry Agent ecosystem.

Question: 15

Deep Dumps

DRAG DROP -

You have a Microsoft Foundry project that contains a customer support agent grounded in internal documentation. After a recent update, users report the following issues:

Some answers are unsupported by retrieved documents.

A small number of responses are flagged for policy violations.

You need to evaluate each issue.

Which observability signals should you use for each issue? To answer, drag the appropriate observability signals to the correct issues. Each observability signal may be used once, more than once, or not at all. You may need to drag the spit bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Observability signals

⋮ Groundedness evaluation metrics

⋮ Latency breakdown traces

⋮ Risk and safety metrics

⋮ Token usage analytics

Answer Area

Unsupported responses:

Observability signal

Policy violations:

Observability signal

Answer:

Observability signals

⋮ Groundedness evaluation metrics

⋮ Latency breakdown traces

⋮ Risk and safety metrics

⋮ Token usage analytics

Answer Area

Unsupported responses:

⋮ Groundedness evaluation metrics

Policy violations:

⋮ Risk and safety metrics

Explanation:

Unsupported responses

Groundedness evaluation metrics

Policy violations:

Risk and safety metrics

Unsupported responses: When an AI model generates responses containing facts or assertions not present in the reference context, it results in an ungrounded or unsupported response (hallucination). Tracking **Groundedness evaluation metrics** assesses the exact ratio of sentences validated by the source text.

Policy violations: These occur when inputs or outputs conflict with regulatory or internal content guidelines (e.g., hate speech, violence, or sexual content). **Risk and safety metrics** directly monitor hits against predefined Azure AI Content Safety filter thresholds.

Question: 16

Deep Dumps

You have a Microsoft Foundry project named Project1 that contains an agent. The agent uses an OpenAPI 3.0 specification to call an external weather service.
The weather service requires a key to be passed in an HTTP header. The key value is stored as a connection in Project1.
You need to ensure that the key value from the connection is included automatically whenever the OpenAPI tool is invoked.
What should you configure in the OpenAPI specification?

A. a header parameter defined for each operation

B. an Azure Key Vault connection

C. an API key security scheme

D. a Bearer token security scheme

Answer: C

Explanation:

Technical Justification for Correct Answer: C

To ensure the key value from the connection is included automatically whenever the OpenAPI tool is invoked for the weather service API call, the most appropriate configuration in the OpenAPI specification is **an API key security scheme**. Here's why:

Correct Answer: C - an API key security scheme

Reason: The API key security scheme in OpenAPI allows for the definition of how an API key is passed (e.g., in a header, query, or cookie). Since the weather service requires the key in an HTTP header, configuring an API key security scheme in the OpenAPI specification enables the automatic inclusion of the key from the connection without needing to redefine it for each operation. This approach is scalable and maintainable, especially if the key needs to be updated, as changes can be made in one place.

Why Other Options are Less Suitable:

A. a header parameter defined for each operation

Inconvenience and Redundancy: Defining a header parameter for each operation leads to redundancy and increased maintenance effort, especially in large APIs. If the key changes, it would need to be updated in multiple places.

Lack of Central Management: Doesn't leverage the connection storage's central management capability efficiently.

B. an Azure Key Vault connection

Misalignment with Requirement: While Azure Key Vault is suitable for securely storing secrets, the question specifies that the key is already stored as a connection in Project1. The task is about integrating this existing setup with the OpenAPI specification, not about where to store the key.

Additional Complexity: Introducing Key Vault at this stage would add unnecessary complexity to the immediate requirement of passing the key in the HTTP header.

D. a Bearer token security scheme

Inappropriate for API Keys: Bearer tokens are typically used for authentication in scenarios involving user authentication or service-to-service calls with more complex authentication flows, not for simple API key exchanges.

Mismatch with Service's Requirement: The weather service specifically requires an API key in the header, not a Bearer token.

References

[OpenAPI Specification - Security Schemes](#)

[Microsoft Azure - Secure your Azure API using API Keys](#)

Question: 17

Deep Dumps

You have a Microsoft Foundry project that serves a high-volume chat app. Most requests are simple FAQs, but some require advanced reasoning.

You need to reduce costs and latency for common queries, without degrading the quality of the responses to

complex questions.
What should you do?

- A. Route all the requests to a smaller model.
- B. Use a model cascade that routes the requests to different models.
- C. Increase the value of the max_tokens parameter for all the requests.
- D. Route all the requests to the most capable model.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the Best Option: Using a model cascade that routes requests to different models is the optimal approach for this scenario. This architecture allows for:

Cost Reduction: Simple FAQs can be handled by smaller, more efficient models with lower inference costs, reducing overall expenses.

Latency Reduction: Routing common queries to lighter models decreases response times for the majority of requests.

Quality Preservation for Complex Queries: Advanced reasoning questions are directed to more capable (likely larger) models, ensuring response quality is maintained without compromise.

Why Other Options are Less Suitable:

A. Route all requests to a smaller model:

Quality Deterioration: Complex questions may not be answered accurately, degrading overall service quality.

Inadequate Capacity: A single smaller model might not handle the "high-volume" aspect efficiently, potentially increasing latency under load.

C. Increase the max_tokens parameter for all requests:

Cost Increase: Higher max_tokens values increase costs per request, contradicting the goal of reducing costs.

Unnecessary Overhead: Simple queries do not benefit from this increase, leading to wasted resources.

D. Route all requests to the most capable model:

Excessive Cost: Utilizing the most capable model for all queries significantly increases costs due to its higher inference expense.

Latency Concerns: Larger models typically have longer response times, affecting the latency of simple, high-volume FAQs.

Correct Answer Justification Summary: Option B, using a model cascade, strikes a balance between cost efficiency, latency reduction for common queries, and maintaining the quality of responses for complex inquiries, making it the most suitable solution.

References:

[Microsoft Azure Documentation: Model Deployment Strategies](#)

[Microsoft Azure Cognitive Services: Optimizing Costs for AI Workloads](#)

Question: 18

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains an internal Q&A agent.

Users report the following issues when they ask the agent questions:

An increase in the following response: "No relevant information found"

Periodic HTTP 429 rate limit exceeded errors during peak hours

You need to identify whether each issue is caused by model unavailability, resource limits, or inference failures. What should you do? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Metrics to enable:

Model Availability Rate and Provisioned Utilization
Only Tokens Cache Match Rate
Only Total Requests filtered to status code 200
Time To Response and Total Tokens

Diagnostic log to collect:

AllMetrics
audit
RequestResponse
trace

Answer:

Answer Area

Metrics to enable:

Model Availability Rate and Provisioned Utilization
Only Tokens Cache Match Rate
Only Total Requests filtered to status code 200
Time To Response and Total Tokens

Diagnostic log to collect:

AllMetrics
audit
RequestResponse
trace

Explanation:

Metrics to enable :

Model Availability Rate and Provisioned Utilization

Model Availability Rate isolates whether service errors or operation drop-offs are caused by downstream, server-side model unavailability.

Provisioned Utilization provides direct observability into capacity and resource limits. When it approaches or exceeds 100%, requests will automatically experience rate-limiting triggers and throw standard HTTP 429 Too Many Requests errors.

Diagnostic log to collect

Request Response.

To store and inspect telemetry data for incoming client interactions, the **RequestResponse** log category explicitly routes full API payloads (including prompts, generated tokens, status codes, and exact completions) to a destination [Log Analytics Workspace](#).

Question: 19

Deep Dumps

You have a Microsoft Foundry project that contains a high-traffic agent. After a recent update, operational costs increase significantly. Monitoring confirms that the volume of user traffic to the agent remains unchanged. You suspect that changes to the request or response characteristics are causing the increase. You need to identify whether the additional costs are driven by the model input size, the model output size, or expanded tool usage. Which observability capability should you use?

- A. latency
- B. evaluation metrics
- C. run success rate
- D. token usage

Answer: D

Explanation:

Technical Justification for Choosing D. Token Usage

To identify the cause of increased operational costs in the Microsoft Foundry project, given that user traffic volume remains unchanged, we need to focus on the changed characteristics of the requests, responses, or tool usage that could drive up costs. Here's why **D. Token Usage** is the most suitable observability capability, alongside explanations for why the other options are less suitable:

D. Token Usage:

Why It's the Best Choice: In the context of AI agents, especially those involving natural language processing (NLP) or similar models, "token usage" refers to the number of tokens (e.g., words, characters) processed by the model in requests and responses. Increased model input size (more tokens in requests) or model output size (more tokens in responses) directly correlates with token usage metrics. Since the question hints at changes in request or response characteristics, monitoring token usage can pinpoint if the cost increase is due to larger inputs, outputs, or potentially more complex queries (initial step before deeper tool usage analysis).

Cost Correlation: Token usage is often directly tied to billing in cloud AI services, as processing more tokens incurs higher costs. This metric can clearly indicate if the increase in costs is due to the model handling more data per interaction.

A. Latency:

Why It's Less Suitable: While latency might indicate performance issues or model complexity increases, it does not directly correlate with cost increases related to input/output size or tool usage. Latency could be affected by many factors unrelated to the suspected causes (e.g., infrastructure, model complexity).

B. Evaluation Metrics:

Why It's Less Suitable: Evaluation metrics (e.g., accuracy, F1 score) are crucial for model performance assessment but do not provide insight into the operational costs driven by input size, output size, or tool usage expansions. These metrics focus on model efficacy, not cost drivers.

C. Run Success Rate:

Why It's Less Suitable: The success rate of model runs indicates the frequency of successful executions but does not offer insights into what drives up costs. A high success rate with increased costs would not help differentiate between input size, output size, or expanded tool usage as the cause.

Conclusion: Given the need to identify cost drivers among model input size, model output size, or expanded tool usage, with the latter two being more directly related to the former, **D. Token Usage** is the most appropriate observability capability. It provides a direct measure of how much data is being processed, which is typically correlated with cost in AI model deployments.

References

1. **Microsoft Azure Cognitive Services Pricing Page:** <https://azure.microsoft.com/en-us/pricing/details/cognitive-services/> - To understand how token usage and other metrics impact billing.
2. **Azure Monitor for Azure Cognitive Services:** <https://docs.microsoft.com/en-us/azure/cognitive-services/cognitive-services-usage-metrics> - For detailed information on available observability metrics, including token usage.

Question: 20

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains an agent.

The agent uses tools to retrieve internal content and call external APIs. The agent is configured to let the model decide when to call the tools.

You need to publish the agent for a compliance workflow. The solution must meet the following requirements:

Each workflow run must include a retrieval step before generating a response.

Tool calls must authenticate by using the published agent's own identity.

Tool access must use an identity isolated from other project resources.

Tool access must use support audit tracing.

What should you do? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Set tool_choice to:

auto
none
required

Configure the tool to authenticate by:

Storing API keys in prompts
Using the shared project agent identity
Using a distinct agent identity bound to the client application

Answer:

Answer Area

Set tool_choice to:

auto
none
required

Configure the tool to authenticate by:

Storing API keys in prompts
Using the shared project agent identity
Using a distinct agent identity bound to the client application

Explanation:

Set tool_choice to

required

Configure the tool to authenticate by:

Using a distinct agent identity bound to the client application

Set tool_choice to (required): Setting this to **required** ensures that the AI model is deterministically forced to call one or more tools before returning a final answer, rather than autonomously deciding whether to skip the tool (as it would with auto).

Configure the tool to authenticate by (Using a distinct agent identity bound to the client application): To meet compliance standards stating that tool access must utilize an identity **isolated from other project resources** and support full audit tracing, assigning a distinct agent identity bound explicitly to the client application ensures complete isolation and individual request tracking.

Question: 21

Deep Dumps

You have a Microsoft Foundry project named Project1 that contains the following:

An OpenAPI tool that calls an external API

A project connection named Connection1 that stores the API key of the external API

When an agent calls the OpenAPI tool, the API returns a 401 unauthorized error, and traces show that the API key header is NOT being sent.

You need to ensure that the OpenAPI tool automatically includes the API key from Connection1 on all requests. What should you do?

- A. Enable identity passthrough so that the tool uses the Microsoft Entra token of the caller.
- B. Add the API key header manually to the OpenAPI specification.
- C. Configure the tool to use the default connection of Project1.
- D. Connect the tool to Connection1.

Answer: D

Explanation:

Technical Justification for Correct Answer D

Correct Answer: D. Connect the tool to Connection1

Connect the tool to Connection1 is the most appropriate solution because it directly addresses the issue by leveraging the existing infrastructure for secure and managed credential handling. Here's why:

Secure Credential Handling: Connection1 already stores the API key. By connecting the tool to this connection, the API key is securely injected into the requests without manual interception, reducing security risks associated with hardcoding or manually handling sensitive information.

Automation and Consistency: This approach ensures the API key is automatically included in all requests to the external API, aligning with the requirement for automation and consistency across all tool invocations.

Management and Updates: If the API key needs to be updated, changes can be made in one place (Connection1), and all connected tools will automatically use the new key without requiring individual updates.

Why Other Options are Less Suitable:

A. Enable Identity Passthrough:

Inapplicable Scenario: Identity passthrough is useful for scenarios where the caller's identity needs to be propagated, not for including a pre-stored API key from a project connection.

Security and Permission Issues: Might introduce unnecessary permissions or complicate access control, as it relies on the caller's Microsoft Entra token rather than a dedicated API key.

B. Add the API Key Header Manually to the OpenAPI Specification:

Security Risk: Hardcoding the API key directly in the OpenAPI spec exposes the key in plain text, posing a significant security risk.

Management Overhead: Updates to the API key would require manual changes to the spec, contradicting best practices for secret management.

C. Configure the Tool to Use the Default Connection of Project1:

Assumes Incorrect Setup: The premise implies that the default connection would contain or automatically provide the necessary API key, which is not specified or guaranteed.

Lacks Explicit Control: Does not ensure the specific API key from Connection1 is used, potentially leading to the same authorization issue if the default connection lacks the key or uses a different one.

References

[Microsoft Azure: Securely store and retrieve secrets with Azure Key Vault](#)

[Microsoft Foundry Documentation: Connecting Tools to Project Connections](#)

Question: 22

Deep Dumps

You have a Microsoft Foundry project that contains a customer support agent. The agent calls an internal knowledge API tool before generating responses.

Users report the following issues:

Some requests take more than 15 seconds to complete.

Some responses are incorrect, even when the knowledge API returns the expected data.

You need to inspect individual agent runs to view the ordered sequence of large language model (LLM) calls, tool invocations, and timing information.

Which observability capability should you use?

- A. token usage
- B. monitoring
- C. safety metrics
- D. tracing

Answer: D

Explanation:

Technical Justification for Choosing D. Tracing

To address the reported issues with the Microsoft Foundry project's customer support agent, the chosen observability capability must provide detailed, sequential insights into the agent's interactions, including Large Language Model (LLM) calls, tool invocations (specifically the internal knowledge API), and precise timing information for each step. Here's why **D. Tracing** is the most suitable option, along with explanations for why the other options are less fitting:

D. Tracing (Correct Answer)

Detailed Sequence Insight: Tracing provides an end-to-end view of the system's behavior for a specific run, showing the ordered sequence of events (LLM calls, API invocations). This is crucial for identifying bottlenecks (e.g., why some requests take more than 15 seconds).

Timing Information: Tracing includes latency data for each step, helping pinpoint where delays occur.

Root Cause Analysis for Incorrect Responses: By examining the exact sequence and output of each LLM call and API invocation, you can identify mismatches between expected and actual API returns and how the agent misinterpreted them.

Why Other Options Are Less Suitable

A. Token Usage

Focus: Primarily monitors resource utilization (e.g., token consumption in LLMs).

Shortcoming: Doesn't offer insights into the sequence of events or timing beyond resource usage, making it inadequate for diagnosing the described issues.

B. Monitoring

Focus: Provides high-level, aggregated views of system performance over time (e.g., error rates, average response times).

Shortcoming: Lacks the granularity needed to inspect individual agent runs and the detailed sequence of events required to solve the specific problems.

C. Safety Metrics

Focus: Concentrates on the safety and fairness aspects of AI model outputs (e.g., bias detection, content policy violations).

Shortcoming: Irrelevant to the operational and sequential debugging needs outlined in the scenario (timing issues and response accuracy tied to API returns).

Conclusion: Given the need to inspect the ordered sequence of events with timing information for individual agent runs to resolve both the latency and accuracy issues, **Tracing (D)** is the most appropriate observability capability.

References

1. **Azure Monitor Distributed Tracing:** <https://learn.microsoft.com/en-us/azure/azure-monitor/app/distributed-tracing>
2. **Microsoft Azure Observability Capabilities:** <https://learn.microsoft.com/en-us/azure/architecture/cloud-adoption/.devops/infrastructure-as-code/observability-patterns>

Question: 23

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses.

You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model.

You need to implement controls to mitigate the risk.

Solution: You configure a prompt shield for user prompts.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Correct Answer: B. No

Goal: Implement controls to mitigate the risk of users uploading unsafe images with hidden instructions to manipulate a multimodal AI generative model.

Solution Analysis: Configuring a Prompt Shield for User Prompts

Why B (No) is Correct:

A **Prompt Shield** is designed to sanitize, filter, or moderate **text-based user inputs** (prompts) to prevent malicious text from influencing the model's behavior.

The identified risk Specifically pertains to **image uploads** with embedded hidden instructions, which a prompt shield does not address since it focuses on text inputs, not image content analysis or processing. Configuring a prompt shield leaves the vulnerability associated with image uploads unmitigated.

Why A (Yes) is Incorrect:

Misalignment with the Risk Type: The solution (prompt shield) does not align with the type of risk identified (unsafe images with hidden instructions), as it is tailored for text-based inputs.

Incomplete Mitigation: Choosing "Yes" would imply that the solution adequately addresses the problem, which it does not, as the image-based vulnerability remains unaddressed.

Alternative Suitable Solutions (Not Provided in the Question but for Context):

Image Content Analysis: Utilize Azure Computer Vision or similar services to analyze uploaded images for unsafe content or hidden instructions.

Image Preprocessing: Implement techniques to sanitize images (e.g., re-encode, thumbnail generation) to remove potential hidden text instructions before processing.

References:

[Azure Computer Vision Overview](#)

[Azure Cognitive Services - Secure Your Resources](#)

Question: 24

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses.

You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model.

You need to implement controls to mitigate the risk.

Solution: You configure image moderation to block unsafe content before processing the images.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Solution Evaluation

Scenario Recap: Mitigate risks in a multimodal AI generative model by preventing unsafe image uploads that embed hidden instructions to manipulate the model.

Proposed Solution: Configure image moderation to block unsafe content before processing the images.

Evaluation of the Solution:

Why the Correct Answer is B (No):

Insufficiency in Addressing Hidden Instructions: Image moderation primarily focuses on identifying and blocking explicit, inappropriate, or unsafe visible content (e.g., nudity, violence). However, it may not effectively detect hidden instructions embedded in images (e.g., steganography, subtle text overlays not immediately recognizable as harmful).

Scope of "Unsafe Content": The solution assumes image moderation's definition of "unsafe" aligns with the threat of embedded instructions, which might not be the case. Embedded instructions could be crafted to avoid moderation flags.

Why Other Options are Not Provided but an Explanation for A (Yes) if it were Considered:

Hypothetical Defense for A (Yes): If the image moderation service used had advanced capabilities explicitly including detection of subtle, embedded textual instructions within images (a less common feature), then A could be justified. However, this is not a standard assumption for most image moderation tools.

Conclusion: Given the standard capabilities of image moderation services, the proposed solution does **not** fully meet the goal of mitigating the risk of embedded hidden instructions in images. Additional or alternative measures (e.g., more sophisticated image analysis for steganography, pre-processing to normalize images before text extraction) would be necessary to address the specified threat comprehensively.

References

[Microsoft Azure Content Moderator Documentation](#): Highlights the capabilities and limitations of standard image moderation services.

[Steganography Detection Techniques](#): Provides insight into the complexities of detecting hidden data in images, underscoring why standard moderation might fail.

Question: 25

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses.

You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model.

You need to implement controls to mitigate the risk.

Solution: You configure a prompt shield for documents.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Correct Answer: B. No

The proposed solution involves configuring a "prompt shield for documents" to mitigate the risk of unsafe images embedding hidden instructions in a multimodal AI generative model. Here's why this solution does not meet the goal and why other implied options (though not listed) might be less suitable in this specific context, followed by an explanation of a more appropriate approach:

Why the Proposed Solution (Prompt Shield for Documents) is Insufficient (B. No):

Misalignment with the Threat Vector: The primary issue is with **image uploads**, not documents. A prompt shield designed for documents would not directly address the vulnerability in processing images, as it doesn't target the image upload and processing pathway.

Technology Misapplication: Prompt shields are typically designed to filter, sanitize, or control text inputs to prevent harmful prompts. Applying this concept to images requires a fundamentally different approach, focusing on image analysis and content filtering, which the proposed solution does not accomplish.

Implicit Discussion on Other Potential Solutions (Not Listed but for Context):

Image Content Filtering/Analysis Services:

Why More Suitable: Directly addresses the image upload issue by analyzing the image content for safety and embedded texts.

Technical Justification: Utilizing AI-powered image analysis services (e.g., Azure Computer Vision, Azure Content Moderator) can detect and filter out unsafe images or extract and sanitize text from images before it reaches the generative model.

Embedded Text Detection in Images:

Why More Suitable: Targets the specific risk of hidden instructions within uploaded images.

Technical Justification: Implementing OCR (Optical Character Recognition) technologies integrated with NLP analysis to identify and neutralize potentially manipulative text within images.

Correct Approach Summary:

To mitigate the risk effectively, the solution should involve **image-centric security measures**, such as integrating image content filtering services or embedded text detection mechanisms, rather than a document-focused prompt shield.

References

For further reading on appropriate technologies to address the image upload security risk:

Azure Content Moderator: <https://azure.microsoft.com/en-us/services/cognitive-services/content-moderator/>

Azure Computer Vision: <https://azure.microsoft.com/en-us/services/cognitive-services/computer-vision/>

Question: 26

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More

than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses.

You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model.

You need to implement controls to mitigate the risk.

Solution: You configure protected material detection.

Does this meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification for Solution Evaluation

Scenario Recap: Mitigate risks associated with unsafe image uploads and hidden instructions in a multimodal AI generative model using image uploads and extracted text for response generation.

Proposed Solution: Configure protected material detection.

Evaluation of Solution against Goal:

Why B. No (Correct Answer)

Protected Material Detection Limitations: Configuring protected material detection is primarily designed to identify and flag copyrighted, sensitive, or explicitly defined "protected" content (e.g., child protection, explicit content). While useful, its core functionality does not specifically target or effectively mitigate the risk of **hidden instructions embedded in images** designed to manipulate the model's output.

Vulnerability to Adversarial Inputs: The solution does not address the root issue of the model's vulnerability to adversarial inputs via image uploads. Protected material detection might not recognize cleverly disguised or technically embedded instructions not classified as "protected" content.

Narrow Scope: This solution focuses on content type rather than the integrity and security of the input data in relation to the model's functionality, leaving the model exposed to the identified manipulation risk.

Why A. Yes is Incorrect

Insufficient Risk Mitigation: Selecting "Yes" implies that protected material detection adequately addresses the specified risk, which it does not, as explained above.

Misalignment with Security Goal: The goal is to prevent model manipulation through hidden image instructions, a challenge that requires a more targeted security or AI model integrity solution.

More Suitable Approaches (Not Listed but for Context)

Image Preprocessing and Sanitization: Techniques to scrub or detect hidden data within images before processing.

Model Input Validation and Parser Security: Enhancing the security of the text extraction process to identify and reject manipulated inputs.

Adversarial Training or Robustness Enhancements: Modifying the model to be more resilient against manipulated inputs.

References

For further reading on securing AI models and image processing security:

Microsoft Azure - Secure Your AI Models: <https://azure.microsoft.com/en-us/services/cognitive-services/security/>

OWASP - Secure Image Uploading: https://owasp.org/www-project-security-cheatsheet/cheatsheet/Validation_Cheat_Sheet#Secure_Image_Uploading

Question: 27

Deep Dumps

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the “Next” button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the “Question” button to return to the question.

Overview -

Company Information -

Contoso, Ltd is a multinational retail company that builds, deploys, and manages generative AI and agent-based solutions by using Microsoft Foundry.

Existing Environment -

Identity Environment -

Contoso uses Microsoft Entra ID for identity management, authentication, and authorization capabilities that enable agents to access organizational resources and services.

Contoso recently formed a new AI engineering team named Agent1Dev Team to optimize and maintain existing AI solutions.

The team collaborates with solution architects, DevOps engineers, and security engineers to design, implement, monitor, and secure AI applications.

Contoso also has a team named Agent1Test Team that is responsible for validating AI solutions before the solution deployments.

Generative Environment -

Contoso has a Microsoft Foundry deployment that contains two projects named Project1 and Project2.

Project1 -

Project1 contains a customer support agent named Agent1 that assists customers with product inquiries and troubleshooting requests.

Agent1 has the following configurations:

Agent1 uses a base model deployment.

A safety evaluation pipeline is NOT enabled.

Tool invocation approval workflows are NOT enabled.

Conversation memory constraints are NOT configured.

Agent1 interacts with customers by using digital support channels and answers general questions about Contoso products.

Project1 is deployed to an Azure region located in the European Union (EU).

Agent1Dev Team will use Project1 to optimize and maintain Agent1.

Project2 -

Project2 contains a deployed video generation model. The marketing department at Contoso has access to Project2 and plans to use the model to develop a video creation solution.

Development of the solution is incomplete.

Data Environment -

Contoso stores product-related information in Azure resources that support AI applications.

The Azure environment contains an Azure Blob Storage account named storage1 that stores product detail sheets for all the Contoso products.

The product sheets include specifications, feature descriptions, and product support information that Agent1 can use to answer customer questions. The product sheets are stored in the PDF format.

Problem Statements -

Contoso identifies the following issues:

Agent1 has only general knowledge of the Contoso products.

A recent chat interaction with Agent1 was analyzed for sentiment. The results of the analysis have NOT been processed yet.

Agent1 does NOT use the detailed product information in the product sheets stored in storage1 when responding to customer questions.

The finance department at Contoso reports that vendor invoices must be reviewed manually to ensure that the invoices match the terms defined in the vendor contracts. The invoices contain tables, logos, and varied layouts that make the documents difficult to process consistently.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement a solution for Project1 that analyzes the vendor invoices by evaluating both the visual layout and the textual content of the invoices, so that the invoice details can be verified against the vendor contract terms.

Update the base model deployment used by Agent1 and standardize the model version to ensure continuity and consistent responses.

Enable Agent1 to retrieve and use the detailed product information from the product sheets stored in storage1.

Implement an indexing solution for the product sheets that Agent1 can use to answer customer questions.

Complete the development of the video creation solution.

Technical Requirements -

Contoso identifies the following technical requirements:

The model deployment used by Agent1 must support scalable, high-throughput generative AI workloads and dynamically scale to handle variable customer support traffic, without requiring reserved throughput capacity.

The product sheets must be processed by using an indexing pipeline that enables semantic and vector search, so that Agent1 can retrieve the relevant product information.

Responses generated by using the product sheet information must be relevant, complete, and accurate.

Agent1 must be able to use the product sheets to answer natural language questions about product details.

The model version used by Agent1 must remain consistent to ensure stable responses.

The data processed by the model must remain within the EU.

Security and Compliance Requirements

Contoso identifies the following security and compliance requirements:

API keys must NOT be used to access Foundry-deployed models.

Access to the Azure resources must follow the principle of least privilege.

The developers at Contoso must authenticate to Microsoft Foundry resources by using Microsoft Entra authentication.

Access to Project1 must be assigned to the members of Agent1Dev Team by using a security group named SC_Agent1_Dev.

Access to Project1 must be assigned to the members of Agent1Test Team by using a security group named SC_Agent1_Test.

Agent1 must never reveal customer information, even if a document that contains customer data is added erroneously to the product sheet repository in storage1.

The product sheets might contain images that include embedded text. Agent1 must be protected from malicious instructions potentially hidden within the images.

Business Requirements -

Contoso identifies the following business requirements:

Users that interact with Agent1 must have a personalized experience in future interactions, including the ability for Agent1 to retain conversation context and recall relevant information from previous interactions.

Agent1 must answer questions only about the products sold by Contoso.

You need to recommend a solution to assess the responses generated by Agent1 when the agent uses the product information stored in storage1. The solution must meet the technical requirements.

What should you include in the recommendation?

- A. a Retrieval Augmented Generation (RAG) evaluator
- B. a custom guardrail
- C. model fine-tuning
- D. a groundedness evaluator

Answer: D

Explanation:

D. a groundedness evaluator.

Why it is correct: A **groundedness evaluator** (such as the GroundednessEvaluator in Azure AI Studio Evaluation) specifically measures how well the model's generated responses align with and are supported by the input source data (the product sheets inside storage1). This assessment ensures that the agent answers questions accurately without fabricating facts or hallucinating information outside the provided company context.

Why the other options are incorrect:

A (RAG evaluator): There is no single generic metric called a "RAG evaluator"; rather, RAG applications are evaluated using a combination of specific modular metrics like groundedness, coherence, and relevance.

B (custom guardrail): Guardrails are real-time execution safety tools (such as [Azure AI Content Safety](#)) used to block policy violations or prompt injections during a live session, not standalone evaluation tools used to retroactively assess/grade generated content quality.

C (model fine-tuning): Fine-tuning is an optimization technique used to customize a model's style, format, or specialized domain vocabulary, not an assessment or evaluation metric solution.

Question: 28

Deep Dumps

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Overview -

Company Information -

Contoso, Ltd is a multinational retail company that builds, deploys, and manages generative AI and agent-based solutions by using Microsoft Foundry.

Existing Environment -

Identity Environment -

Contoso uses Microsoft Entra ID for identity management, authentication, and authorization capabilities that enable agents to access organizational resources and services.

Contoso recently formed a new AI engineering team named Agent1Dev Team to optimize and maintain existing AI solutions.

The team collaborates with solution architects, DevOps engineers, and security engineers to design, implement, monitor, and secure AI applications.

Contoso also has a team named Agent1Test Team that is responsible for validating AI solutions before the solution deployments.

Generative Environment -

Contoso has a Microsoft Foundry deployment that contains two projects named Project1 and Project2.

Project1 -

Project1 contains a customer support agent named Agent1 that assists customers with product inquiries and troubleshooting requests.

Agent1 has the following configurations:

Agent1 uses a base model deployment.

A safety evaluation pipeline is NOT enabled.

Tool invocation approval workflows are NOT enabled.

Conversation memory constraints are NOT configured.

Agent1 interacts with customers by using digital support channels and answers general questions about Contoso products.

Project1 is deployed to an Azure region located in the European Union (EU).

Agent1Dev Team will use Project1 to optimize and maintain Agent1.

Project2 -

Project2 contains a deployed video generation model. The marketing department at Contoso has access to Project2 and plans to use the model to develop a video creation solution.

Development of the solution is incomplete.

Data Environment -

Contoso stores product-related information in Azure resources that support AI applications.

The Azure environment contains an Azure Blob Storage account named storage1 that stores product detail sheets for all the Contoso products.

The product sheets include specifications, feature descriptions, and product support information that Agent1 can use to answer customer questions. The product sheets are stored in the PDF format.

Problem Statements -

Contoso identifies the following issues:

Agent1 has only general knowledge of the Contoso products.

A recent chat interaction with Agent1 was analyzed for sentiment. The results of the analysis have NOT been processed yet.

Agent1 does NOT use the detailed product information in the product sheets stored in storage1 when responding to customer questions.

The finance department at Contoso reports that vendor invoices must be reviewed manually to ensure that the invoices match the terms defined in the vendor contracts. The invoices contain tables, logos, and varied layouts that make the documents difficult to process consistently.

Requirements -

Planned Changes -

Contoso plans to implement the following changes:

Implement a solution for Project1 that analyzes the vendor invoices by evaluating both the visual layout and the textual content of the invoices, so that the invoice details can be verified against the vendor contract terms.

Update the base model deployment used by Agent1 and standardize the model version to ensure continuity and consistent responses.

Enable Agent1 to retrieve and use the detailed product information from the product sheets stored in storage1.

Implement an indexing solution for the product sheets that Agent1 can use to answer customer questions.

Complete the development of the video creation solution.

Technical Requirements -

Contoso identifies the following technical requirements:

The model deployment used by Agent1 must support scalable, high-throughput generative AI workloads and dynamically scale to handle variable customer support traffic, without requiring reserved throughput capacity.

The product sheets must be processed by using an indexing pipeline that enables semantic and vector search, so that Agent1 can retrieve the relevant product information.

Responses generated by using the product sheet information must be relevant, complete, and accurate.

Agent1 must be able to use the product sheets to answer natural language questions about product details.

The model version used by Agent1 must remain consistent to ensure stable responses.

The data processed by the model must remain within the EU.

Security and Compliance Requirements

Contoso identifies the following security and compliance requirements:

API keys must NOT be used to access Foundry-deployed models.

Access to the Azure resources must follow the principle of least privilege.

The developers at Contoso must authenticate to Microsoft Foundry resources by using Microsoft Entra authentication.

Access to Project1 must be assigned to the members of Agent1Dev Team by using a security group named SC_Agent1_Dev.

Access to Project1 must be assigned to the members of Agent1Test Team by using a security group named SC_Agent1_Test.

Agent1 must never reveal customer information, even if a document that contains customer data is added erroneously to the product sheet repository in storage1.

The product sheets might contain images that include embedded text. Agent1 must be protected from malicious instructions potentially hidden within the images.

Business Requirements -

Contoso identifies the following business requirements:

Users that interact with Agent1 must have a personalized experience in future interactions, including the ability for Agent1 to retain conversation context and recall relevant information from previous interactions.

Agent1 must answer questions only about the products sold by Contoso.
You need to configure Agent1 to answer customer questions about only the Contoso products. The solution must meet the business requirements.
What should you do?

- A. Modify the system message instructions.
- B. Add few-shot examples.
- C. Apply top-p sampling.
- D. Increase the value of the temperature parameter.

Answer: A

Explanation:

Technical Justification for Correct Answer: A. Modify the system message instructions

To address the business requirement that "Agent1 must answer questions only about the products sold by Contoso," modifying the system message instructions is the most appropriate action. Here's why:

Why A is the best choice:

System Message Instructions are used to provide initial or contextual instructions to the model, defining its scope, behavior, or the domain of knowledge it should operate within. By modifying these instructions, you can explicitly limit Agent1's response domain to Contoso's products, aligning with the business requirement for personalized and relevant interactions.

This approach directly influences the model's understanding of its operational boundaries without altering the underlying model's capabilities unnecessarily.

Why other options are less suitable:

B. Add few-shot examples:

While few-shot examples can guide the model towards understanding the desired output for specific inputs, they do not guarantee a strict limitation to Contoso products across all possible queries. This method is more about fine-tuning for specific examples rather than setting a broad operational boundary.

C. Apply top-p sampling:

Top-p sampling is a technique to control the diversity of generated text by only considering the top probabilities. It does not restrict the model's domain knowledge or ensure responses are limited to Contoso products; it merely influences the variability of responses.

D. Increase the value of the temperature parameter:

The temperature parameter in generation models controls the randomness of the output. Increasing it would lead to more diverse but potentially less relevant or accurate responses. This has no direct bearing on limiting the model's responses to Contoso products.

Additional Considerations Aligning with Other Requirements:

For **scalable, high-throughput generative AI workloads** and **dynamic scaling** without reserved throughput, leveraging Azure's auto-scaling capabilities for the model deployment (possibly through Azure Kubernetes Service (AKS) or serverless options like Azure Functions) would be advisable, though this is not directly related to the question asked.

Indexing solution for semantic and vector search of product sheets could be achieved with Azure Cognitive Search, integrating with the model to fetch relevant product information.

Consistency in model version can be managed through careful deployment strategies and version control in

Microsoft Foundry.

Data processed within the EU is already addressed by Project1's deployment in an EU Azure region.

Security Requirements (least privilege, Microsoft Entra authentication, etc.) are met through the described use of security groups and avoiding API keys, aligning with the provided security and compliance requirements.

References

1. **Microsoft Learn - Customize model behavior with system messages:** <https://learn.microsoft.com/en-us/azure/cognitive-services/language-generation/customize-model-behavior-system-messages>
2. **Azure Cognitive Search for Semantic Search:** <https://learn.microsoft.com/en-us/azure/search/search-what-is-semantic-search>

Question: 29

Deep Dumps

You have a Microsoft Foundry project.

You plan to build a customer support solution that contains an agent. The solution must meet the following requirements:

Provide accurate, context-aware responses grounded in internal product documentation stored in Azure AI Search.

Require deep, multi-step reasoning across long contexts.

Generate detailed natural language responses.

Which type of model should you use to power the agent?

- A. a multimodal model
- B. a small language model (SLM)
- C. a key phrase extraction model
- D. a large language model (LLM)

Answer: D

Explanation:

Technical Justification for Choosing D: a Large Language Model (LLM)

Requirements Analysis and Model Selection

The customer support solution requires a model that can:

1. Provide accurate, context-aware responses based on internal product documentation stored in Azure AI Search.
2. Perform deep, multi-step reasoning across long contexts.
3. Generate detailed natural language responses.

Why D (Large Language Model - LLM) is the Best Choice:

Context Awareness and Accuracy: LLMs are trained on vast amounts of text data, enabling them to understand complex contexts and provide accurate responses. Integrating with Azure AI Search, an LLM can effectively leverage the internal product documentation to generate informed responses.

Deep, Multi-Step Reasoning: The architectural design of LLMs, often based on transformer models, facilitates the handling of long-range dependencies and multi-step reasoning, crucial for resolving complex customer inquiries.

Detailed Natural Language Responses: LLMs are renowned for their capability to produce coherent, detailed, and natural-sounding text, aligning perfectly with the need for comprehensive support responses.

Why Other Options are Less Suitable:

A. Multimodal Model:

While useful for processing multiple data types (e.g., text, images), the primary requirement here focuses on text-based reasoning and response generation, making the multimodal aspect less critical.

May introduce unnecessary complexity without direct benefit for the specified text-centric tasks.

B. Small Language Model (SLM):

SLMs lack the scale and depth of training data compared to LLMs, potentially resulting in less accurate or less contextually aware responses.

May struggle with the deep, multi-step reasoning required for complex support queries.

C. Key Phrase Extraction Model:

Designed primarily for identifying key phrases rather than generating detailed, context-aware responses.

Fails to meet the requirements for multi-step reasoning and natural language response generation.

Conclusion Given the specific requirements of the customer support solution, a **Large Language Model (LLM)** is the most appropriate choice due to its capabilities in context-aware response generation, deep reasoning, and detailed natural language output.

References

1. **Azure AI Search Documentation:** <https://learn.microsoft.com/en-us/azure/search/search-what-is-azure-search>
2. **Microsoft Learn - Large Language Models:** <https://learn.microsoft.com/en-us/azure/cognitive-services/language-service/concepts/large-language-models>

Question: 30

Deep Dumps

DRAG DROP -

You have a Microsoft Foundry project that contains a deployed ticket-triage agent.

You discover that sometimes the agent responds without calling any tools, even when a tool is required.

You need to ensure that the agent calls a tool during execution.

How should you complete the Python code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

⋮

"auto"

⋮

"required"

⋮

"response_format"

⋮

"tool_choice"

⋮

"tools"

⋮

"type"

Answer Area

```
run_payload = {  
    "assistant_id": agent_id,  
    Value : Value ,  
    "metadata": {  
        "scenario": "ticket-triage"  
    }  
}
```

Answer:

Values

:: "auto"

:: "required"

:: "response_format"

:: "tool_choice"

:: "tools"

:: "type"

Answer Area

```
run_payload = {  
    "assistant_id": agent_id,  
    "tool_choice" : "required",  
    "metadata": {  
        "scenario": "ticket-triage"  
    }  
}
```

Explanation:

"tool_choice": This parameter key overrides the default tool execution behavior for a specific thread run.

required: Setting this value forces the agent to deterministically invoke at least one configured tool (such as the MCP tool mentioned in prior scenario requirements) before providing a final response, preventing the agent from bypassing tool execution.

Question: 31

Deep Dumps

You are building a web app named App1 that generates responses by using a model deployed to a Microsoft Foundry project named Project1.

Before sending the prompts to the model, App1 must retrieve documents by using Azure AI Search.

You need to integrate Project1 and App1. The solution must meet the following requirements:

Multiple client applications must use the same search configuration.

A security policy must prevent key-based authentication.

Administrative effort must be minimized.

What should you do?

- A. Create a custom HTTP connection in Foundry and manually configure Azure AI Search endpoints per application.
- B. Configure an Azure AI Search connection in Project1 and reference the connection in each application.
- C. Call Azure AI Search directly from each application by using Microsoft Entra authentication.
- D. Enable a managed identity for each application and call Azure AI Search directly.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the Best Option:

Meets Requirement 1: Multiple Client Applications - Configuring an Azure AI Search connection in Project1 allows for a single, reusable search configuration that can be easily referenced by multiple client applications (including App1), streamlining the setup process.

Meets Requirement 2: Prevents Key-Based Authentication - By integrating the search connection at the Project1 level, the solution can leverage more secure authentication methods (e.g., managed identities, service principals with appropriate permissions) instead of key-based authentication for Azure AI Search.

Meets Requirement 3: Minimizes Administrative Effort - Centralizing the Azure AI Search configuration in Project1 reduces the administrative burden compared to configuring each application individually. Updates or

changes to the search configuration can be made in one place.

Why Other Options are Less Suitable:

A. Custom HTTP Connection in Foundry (Manual Configuration per Application)

Fails Requirement 3: Maximal administrative effort due to manual configuration for each application.

Inefficient: Does not scale well with multiple applications.

C. Call Azure AI Search Directly from Each Application using Microsoft Entra Authentication

Partially Addresses Requirement 2: While Entra authentication is secure, this option does not inherently prevent key-based authentication if not properly configured.

Fails Requirement 1 & 3: Each application would need its own configuration, increasing effort and potentially leading to configuration inconsistencies.

D. Enable Managed Identity for Each Application and Call Azure AI Search Directly

Addresses Requirement 2: Secure authentication via managed identities.

Fails Requirement 1 & 3: Similar to C, this approach requires per-application setup and maintenance, lacking the centralization benefit for the search configuration.

Correct Action: B. Configure an Azure AI Search connection in Project1 and reference the connection in each application.

References

1. **Azure AI Search Documentation** - [Integrate Azure AI Search with Azure Services](#)
2. **Microsoft Foundry and Azure AI Search Integration** - [Deploy and integrate models with Azure services](#) (Refer to sections relevant to Azure AI Search and project integrations)

Question: 32

Deep Dumps

DRAG DROP -

You have a Microsoft Foundry project that contains an agent used by the financial analysts at your company. You need to optimize the agent workflow by providing additional data access and processing capabilities. The solution must meet the following requirements:

Ensure that the agent can perform calculations during conversations.

Ensure that the agent can access up-to-date information from public websites.

Ensure that the agent can retrieve information from documents uploaded directly to the agent.

What should you use for each requirement? To answer, drag the appropriate tools to the correct requirements. Each tool may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Tools	Answer Area
Code interpreter	Access up-to-date information from public websites: Tool
Computer use	Perform calculations during conversations: Tool
File search	Retrieve information from documents uploaded directly to the agent: Tool
Grounding with Bing Search	
Microsoft Fabric	

Answer:

Tools

Code interpreter

Computer use

File search

Grounding with Bing Search

Microsoft Fabric

Answer Area

Access up-to-date information from public websites:

Grounding with Bing Search

Perform calculations during conversations:

Code interpreter

Retrieve information from documents uploaded directly to the agent:

File search

Explanation:

Access up-to-date information (Grounding with Bing Search): This native tool allows the agent to execute real-time web queries to retrieve current information and facts from public websites, mitigating static knowledge-cutoff limitations.

Perform calculations (Code interpreter): This tool creates a sandboxed runtime environment where the agent can dynamically write and execute Python code to handle complex mathematical operations, logical calculations, and data processing accurately.

Retrieve information from documents (File search): This built-in vector search/RAG capability parses and processes local document uploads (such as PDFs, DOCX, or TXT files) directly attached to the agent or thread, allowing the model to quickly search across their contents.

Question: 33

Deep Dumps

You have a Microsoft Foundry project that contains a prompt agent used by a customer support web app. The agent is invoked from a Python service that does NOT run in the Foundry portal. You need to implement end-to-end tracing to capture latency breakdowns and exceptions across agent runs. Which two components can you use? Each correct answer presents a complete solution.
NOTE: Each correct selection is worth one point.

- A. a Log Analytics workspace
- B. Application Insights
- C. OpenTelemetry
- D. the Azure Monitor Agent
- E. Microsoft Sentinel

Answer: BC

Explanation:

Technical Justification for Correct Answer: BC

To implement end-to-end tracing for capturing latency breakdowns and exceptions across agent runs in a Microsoft Foundry project, where the agent is invoked from an external Python service, the chosen components must effectively handle distributed tracing, support integration with both Foundry and non-Azure (or external) services, and provide comprehensive insights into performance and errors.

Why B (Application Insights) is Correct:

Integration with Foundry and External Services: Application Insights (AI) seamlessly integrates with Azure services, including Foundry, and can be easily set up for external applications through its SDKs, which include a Python SDK for the external Python service.

End-to-End Tracing Capability: AI offers automatic detection of dependencies and detailed latency breakdowns, crucial for identifying bottlenecks across the agent and the Python service.

Exception Tracking: It provides robust exception tracking, alerted and detailed enough for debugging across different components of the application.

Why C (OpenTelemetry) is Correct:

Standard for Distributed Tracing: OpenTelemetry is an open standard for distributed tracing and telemetry, making it highly compatible with both Azure services (like Foundry) and external services (like the Python application), ensuring a unified view of the system.

Flexibility in Backends: While OpenTelemetry itself doesn't store data, it can be configured to send traces to various backends, including Application Insights, making it a complementary choice for ensuring traces from all parts of the system are captured consistently.

Python Support: OpenTelemetry has a Python SDK, facilitating its integration with the external Python service for comprehensive tracing.

Why Other Options are Less Suitable:

A. Log Analytics workspace: While useful for log aggregation and analysis, it's more focused on logging than on the end-to-end tracing and latency breakdowns required. It can be used in conjunction with other tools but doesn't provide the tracing capabilities on its own.

D. the Azure Monitor Agent: Primarily designed for collecting metrics and logs from VMs and physical servers, it's not the best fit for tracing in serverless web apps or external services like the described Python service.

E. Microsoft Sentinel: A security information and event management (SIEM) system, it's more focused on security analytics rather than application performance tracing and latency analysis.

References:

1. **Application Insights Documentation:** <https://docs.microsoft.com/en-us/azure/azure-monitor/app/>
2. **OpenTelemetry Documentation (Python):** <https://opentelemetry.io/docs/instrumentation/python/getting-started/>

Question: 34

Deep Dumps

You have a customer support agent that uses the Microsoft Foundry Agent Service.

Sometimes, customers return to a session days later to continue the same support case, and the agent must resume with the full historical context. The agent must provide the following:

Multi-turn continuity within the session

Cross-session continuity for the same case

Access to the full interaction history, including user messages, agent messages, tool calls, and tool outputs

You need to ensure that the agent automatically reloads the complete history on each new turn.

What should you do?

- A. Create and reuse a conversation by storing the conversation's ID and supplying the ID on subsequent requests.
- B. Persist only the final model response stored in the client application and prepend the response to future prompts.
- C. Enable memory summarization on the agent definition to persist the context automatically.

Answer: A

Explanation:

Technical Justification for Correct Option (A)

To ensure the Microsoft Foundry Agent Service automatically reloads the complete history on each new turn for resuming support cases with full historical context, **Option A** is the most suitable choice. Here's why:

Why A is Correct:

Multi-turn and Cross-session Continuity: Creating and reusing a conversation by storing the conversation's ID allows for both multi-turn continuity within a session and cross-session continuity for the same case. This approach maintains the conversation's state across different interactions.

Access to Full Interaction History: By supplying the stored conversation ID on subsequent requests, the agent can retrieve the entire interaction history, including user messages, agent messages, tool calls, and tool outputs, ensuring the agent has the full context.

Automatic Reload of History: This method inherently enables the automatic reloading of the conversation's history upon providing the ID, meeting the requirement without needing additional logic for history retrieval.

Why Other Options are Less Suitable:

B. Persist only the final model response:

Inadequate Context: Only storing the final model response would lose the detailed interaction history, making it impossible to fully resume the case with all context.

Lacks Multi-turn Continuity: Prepending the final response to future prompts does not recreate the conversational flow or provide access to all previous messages and tool interactions.

C. Enable memory summarization on the agent definition:

Summarization Limitations: Memory summarization might not capture the entirety of the interaction history with the same fidelity as storing the conversation ID. It's designed for highlighting key points rather than preserving a complete record.

Unclear Persistence Across Sessions: There's no clear indication that memory summarization persists across sessions in the same detailed manner as reusing a conversation ID. It's more suited for in-session context management.

References

[Microsoft Documentation: Conversations in Microsoft Bot Framework](#)

[Microsoft Foundry Documentation: Managing Conversation State](#) **Note:** Due to the dynamic nature of documentation and the specificity of "Microsoft Foundry Agent Service" (which might be a hypothetical or less-documented service in the provided context), the second link is assumed based on common Microsoft service patterns. For actual implementation, always refer to the most current and specific documentation for your service.

Question: 35

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains a deployed chat model.

You have a Python service that sends API requests to the model. The service is integrated with an automated validation system that compares generated outputs against approved response patterns.

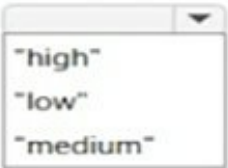
Stakeholders report that small wording differences are causing validation mismatches.

You need to update the request parameters to improve output stability. The solution must maximize reasoning quality.

How should you complete the Python code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
message = client.messages.create(  
    model="deployment-name",  
    messages=[  
        {"role": "user", "content": "Summarize the release notes in 3 bullet points."}  
    ],  
    max_tokens=800,  
    temperature=  
        ,  
    thinking={"type": "enabled"},  
    output_config={"effort":  
          
    }  
)
```

Answer:

Answer Area

```
message = client.messages.create(  
    model="deployment-name",  
    messages=[  
        {"role": "user", "content": "Summarize the release notes in 3 bullet points."}  
    ],  
    max_tokens=800,  
    temperature=  
        ,  
    thinking={"type": "enabled"},  
    output_config={"effort":  
          
    }  
)
```

Explanation:

temperature :1

When enabling extended or adaptive reasoning via the thinking parameter ("type": "enabled"), API rules mandate that the temperature must be set to exactly 1 (or omitted completely). Attempting to use a deterministic value like 0 or high variance like 2 will result in a standard HTTP 400 Bad Request validation error from the API gateway.

output_config= "effort": ... ("low"):

The user request is a straightforward text summary ("Summarize the release notes in 3 bullet points"). To prevent the reasoning model from unnecessarily burning through excessive internal thinking tokens on simple

retrieval/summarization tasks — which directly impacts latency and billing — the overall reasoning intensity should be set to a lower tier like "low" or "medium".

Question: 36

Deep Dumps

You have a chat app in a Microsoft Foundry project and an Azure AI Search vectorized index.

You need to connect to the index to meet the following requirements:

Complex questions must retrieve information from multiple chunks.

Multi-turn conversations must influence retrieval planning.

Retrievals must run in parallel to reduce latency.

Which retrieval approach should you use?

- A. iterative retrieval
- B. agentic Retrieval Augmented Generation (RAG)
- C. chain of thought
- D. classic Retrieval Augmented Generation (RAG)

Answer: B

Explanation:

Technical Justification for Correct Answer: B - Agentic Retrieval Augmented Generation (RAG)

Why B (Agentic Retrieval Augmented Generation) is the Best Choice:

Complex Questions & Multi-Chunk Retrieval: Agentic RAG is designed to handle complex, open-ended questions by leveraging an agent's capability to iteratively query a knowledge base (in this case, the Azure AI Search vectorized index) across multiple chunks of information. This aligns perfectly with the requirement to retrieve information from multiple chunks.

Multi-Turn Conversations Influence: The "agentic" aspect implies the use of an AI agent that can understand context and adjust its queries based on previous conversation turns, directly influencing retrieval planning to provide more accurate and contextual responses.

Parallel Retrievals for Latency Reduction: Agentic RAG, being part of more advanced AI architectures, is more likely to support or be compatible with parallel processing of queries to the vectorized index, reducing latency in retrieving information from multiple sources simultaneously.

Why Other Options are Less Suitable:

A. Iterative Retrieval:

Lack of Contextual Understanding: Iterative retrieval lacks the contextual understanding and adaptive query generation provided by an agentic approach, making it less suitable for multi-turn conversations.

Scalability and Latency: While it can retrieve from multiple chunks, it doesn't inherently support parallel retrievals as effectively as agentic models might, potentially leading to higher latency.

C. Chain of Thought:

Primary Focus: More focused on generating a step-by-step reasoning process for a model's answer rather than efficient, parallel retrieval from an external index.

Not Designed for External Index Interaction: Less optimized for interacting with an external vectorized index in the manner required.

D. Classic Retrieval Augmented Generation (RAG):

Single-Pass Retrieval: Classic RAG typically involves a single retrieval pass, which is less effective for complex, multi-chunk queries compared to the iterative, adaptive approach of Agentic RAG.

Limited Contextual Adaptation: Lacks the agentic capability to adapt retrieval plans based on multi-turn conversation context.

References

[Microsoft Azure - Azure Cognitive Search for Complex Queries](#)

[Research on Agentic Retrieval Augmented Generation for Conversational AI](#)

Question: 37

Deep Dumps

HOTSPOT -

You need to recommend a plan to create a customer support agent by using the Microsoft Foundry Agent Service. The agent must meet the following requirements:

Retain user preferences across multiple conversations.

Enable users to provide contextual grounding by directly uploading documents during a chat.

Which Foundry capability should you recommend for each requirement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

To retain user preferences across conversations, use:

Agent memory that uses persistent storage
Conversation history
Orchestration-managed session context

To enable users to provide contextual grounding during chats, use the:

Azure AI Search tool
Code interpreter tool
File search tool

Answer:

Answer Area

To retain user preferences across conversations, use:

Agent memory that uses persistent storage
Conversation history
Orchestration-managed session context

To enable users to provide contextual grounding during chats, use the:

Azure AI Search tool
Code interpreter tool
File search tool

Explanation:

To retain user preferences across conversations, use:

Agent memory that uses persistent storage.

To enable users to provide contextual grounding during chats, use the:

File search tool

Retaining Preferences Across Conversations: Standard conversation history only persists context within a

single active thread session. To remember user preferences and profile parameters long-term across separate conversations or multiple threads, you must utilize **Agent memory that uses persistent storage** (such as Azure AI Foundry's native persistent memory or long-term profile storage capabilities).

Contextual Grounding During Chats: The **File search tool** allows end users to upload documentation, images, or reference files directly into an active chat session or thread. The agent automatically ingests these documents to provide immediate, contextual grounding for user queries.

Question: 38

Deep Dumps

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents.
You need to improve response completeness. The solution must be implemented in the logic of the application code before responses are returned.
What should you do?

- A. Add a retry evaluation before the responses are returned.
- B. Decrease the value of the max_tokens parameter.
- C. Switch to Retrieval Augmented Generation (RAG).
- D. Replace the model with a smaller deployment.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A is the best option: To improve response completeness in the context of an agent generating summaries from retrieved policy documents within a Microsoft Foundry project, **adding a retry evaluation before responses are returned (Option A)** is the most appropriate solution. This approach directly addresses potential issues with incomplete responses by allowing the system to re-attempt the generation or retrieval process if the initial response does not meet a defined completeness threshold. This can be implemented within the application logic to ensure that responses are thoroughly reviewed before being returned to the user, enhancing overall completeness without necessarily altering the core model or parameters in a way that might compromise other aspects of performance.

Why other options are less suitable:

B. Decrease the value of the max_tokens parameter: Decreasing max_tokens would actually limit the potential length of the responses, which could worsen the issue of incompleteness by artificially capping the response size, potentially truncating summaries.

C. Switch to Retrieval Augmented Generation (RAG): While RAG can enhance the accuracy and relevance of generated text by augmenting the model with external knowledge retrieval, switching to RAG is a significant architectural change. It does not directly address the issue of response completeness in the context of the current implementation's logic before responses are returned. It's more about improving accuracy and relevance than ensuring completeness of the generated summaries based on the retrieved documents.

D. Replace the model with a smaller deployment: Deploying a smaller model could reduce computational costs and potentially speed up response times, but it likely compromises the model's capacity to generate complete and detailed summaries due to reduced computational power and possibly less training data capacity, potentially worsening the completeness issue.

References:

Microsoft Azure Documentation - Retry Logic in Azure Applications: <https://learn.microsoft.com/en-us/azure/architecture/resilience/retry-service-specific>

Azure Cognitive Services - Understanding Model Parameters (max_tokens context):

<https://learn.microsoft.com/en-us/azure/cognitive-services/language/understand-how-to-use-language-model#model-parameters>

Question: 39

Deep Dumps

You have a customer support agent built by using the Microsoft Foundry Agent Service. The agent calls an Azure OpenAI model deployment.

During load testing, calls intermittently fail and return an HTTP 429 rate limit exceeded error.

You need to handle throttling to reduce call failures and improve reliability under load. The solution must remain within the service and model limits.

What should you do?

- A. Create a new thread and retry the calls immediately.
- B. Reduce the number of registered tools.
- C. Implement a retry policy that uses exponential backoff and jitter.
- D. Split uploaded content into smaller files.

Answer: C

Explanation:

Technical Justification for Correct Answer (C)

Why C is the Best Option: Implementing a retry policy with exponential backoff and jitter (Option C) is the most suitable solution for handling throttling and reducing call failures when encountering HTTP 429 rate limit exceeded errors. This approach directly addresses the rate limiting issue by adjusting the retry timing based on the failure response, preventing immediate repeated failures, and reducing the load on the Azure OpenAI model deployment. Exponential backoff increases the wait time between retries after each failure, while jitter introduces randomness in the wait time, preventing a "sudden death" scenario where all retries happen at once after the same wait period, thus more effectively distributing the load.

Why Other Options are Less Suitable:

A. Create a new thread and retry the calls immediately:

Immediate retries without a backoff strategy will likely result in repeated failures, as the rate limit condition hasn't changed.

Creating a new thread for each retry can lead to resource exhaustion and does not address the underlying throttling issue.

B. Reduce the number of registered tools:

This might indirectly reduce the load but does not **directly address the throttling issue** during peak loads. It could lead to **underutilization** of resources if not carefully planned.

D. Split uploaded content into smaller files:

This approach **does not address rate limiting** on API calls to the Azure OpenAI model.

It might be beneficial for upload limits or processing time but is **irrelevant to the throttling problem**

described.

Correct Approach (Detailed):

Implement a Retry Policy with:

Exponential Backoff: Increase the wait time exponentially after each failed retry (e.g., 1s, 2s, 4s, 8s, ...).

Jitter: Introduce randomness in the wait time (e.g., $\text{wait_time} * \text{random_factor}$) to prevent synchronized retries.

Example Implementation Consideration: Utilize Azure's built-in retry mechanisms or libraries (like Polly for .NET) that support exponential backoff with jitter for calls to the Azure OpenAI model, ensuring the solution remains within service and model limits.

References

1. **Azure Documentation - Retry with Exponential Backoff:** <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry-with-exponential-backoff>
2. **Polly (for .NET) - Implementation of Retry with Backoff and Jitter:** <https://github.com/App-vNext/Polly/wiki/Retry-Policy#with-backoff-and-jitter>

Question: 40

Deep Dumps

HOTSPOT -

You have a Microsoft Foundry project that contains an agent.

You use a GitHub Actions workflow for CI/CD.

You need to configure the workflow to automatically evaluate the agent when a pull request (PR) is created and prevent branches from merging if the evaluation results do NOT meet the defined thresholds.

How should you configure the workflow? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

Authentication method:

- A personal access token (PAT)
- A user-assigned managed identity
- An Azure Login action that uses OpenID Connect (OIDC)

If the evaluation results are **NOT** met, configure the workflow to:

- Lock the target branch
- Send an alert
- Fail

Answer:

Answer Area

Authentication method:

A personal access token (PAT)
A user-assigned managed identity
An Azure Login action that uses OpenID Connect (OIDC)

If the evaluation results are **NOT** met, configure the workflow to:

Lock the target branch
Send an alert
Fail

Explanation:

Authentication method

An Azure Login action that uses OpenID Connect (OIDC)

When connecting an automated external runner or automation pipeline to Azure resources, storing long-lived credentials (like [Personal Access Tokens \(PATs\)](#)) introduces security risks. Utilizing an Azure Login action paired with **OpenID Connect (OIDC)** allows for short-lived, credential-less authentication, removing the need to manage secret keys or rotate expirations.

Workflow Action On Failure

Fail.

In evaluation gates (such as checking prompt flow outputs against accuracy or content safety thresholds), if the minimum criteria or evaluation benchmarks are **not met**, the runner pipeline must explicitly **Fail**. This structural break blocks bad builds or ungrounded model revisions from progressing through subsequent stages or merging into the primary deployment branch.

Question: 41

Deep Dumps

Note: This section contains one or more sets of questions with the same scenario and problem. Each question presents a unique solution to the problem. You must determine whether the solution meets the stated goals. More than one solution in the set might solve the problem. It is also possible that none of the solutions in the set solve the problem.

After you answer a question in this section, you will NOT be able to return. As a result, these questions do not appear on the Review Screen.

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents.

Users report that some responses omit required regulatory clauses, even when the clauses are present in the retrieved content.

You need to improve response completeness.

Solution: You increase the value of the max_tokens parameter.

Does this meet the goal?

- A. Yes
- B. No

Answer: A

Explanation:

Technical Justification for Solution Review

Scenario Recap: Enhance response completeness for a Microsoft Foundry agent generating summaries from policy documents, addressing omissions of required regulatory clauses.

Proposed Solution: Increase the value of the max_tokens parameter.

Evaluation and Justification for Correctness of Solution A (Yes)

Rationale for Correctness (A. Yes):

Understanding max_tokens: The max_tokens parameter in the context of AI models (like those used in Microsoft Foundry for text summarization) dictates the maximum number of tokens (or "words/tokens" depending on the model's tokenization scheme) the model can output in a single response.

Impact on Response Completeness: Increasing max_tokens allows the model to generate longer responses. This adjustment directly addresses the issue of omitted regulatory clauses by providing the model with the capacity to include more content from the source document in its summary, thereby potentially increasing the completeness of responses regarding required clauses.

Alignment with Goal: The primary goal is to improve response completeness, especially concerning the inclusion of all necessary regulatory clauses. Increasing max_tokens is a direct and effective strategy to achieve this, making the solution **A (Yes)** correct.

Why Other Options are Less Suitable (B. No)

Rationale Against B (No):

Lack of Justification for Ineffectiveness: Without specific constraints (e.g., performance, response time limits), decreasing or not adjusting max_tokens would not logically address the omission issue. The problem statement does not mention such constraints, making **B (No)** less justifiable without additional context suggesting why increasing max_tokens wouldn't work.

Misalignment with Observed Issue: Saying "No" implies the solution doesn't meet the goal without providing an alternative reason related to the max_tokens adjustment's efficacy in this context, which is technically sound for addressing response length and thus completeness.

Conclusion: Given the technical rationale, increasing max_tokens is a suitable solution to enhance response completeness by allowing for longer summaries that can include previously omitted regulatory clauses. Thus, **A. Yes** is the correct choice.

References

1. **Microsoft Azure Documentation - Customizing Model Output:** <https://docs.microsoft.com/en-us/azure/cognitive-services/language/understanding/how-to-use-BaseModel?tabs=csharp#customize-model-output> (Navigate to sections discussing output controls like max_tokens for relevant model configurations.)
2. **Transformer Model Tokenization Explanation (General AI Context):** <https://huggingface.co/docs/transformers/glossary#tokenization> (While not Azure-specific, provides insight into tokenization and max_tokens implications in AI model contexts.)

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

I wanted to let you know that there is more content available in the full version. The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

[Download Full Version Now](#)



Future is Secured
100% Pass Guarantee



24/7 Customer Support
Mail us - support@deepdumps.com



Verified Updates
Lifetime Updates!

Total: **135 Questions**

Link: <https://deepdumps.com/papers/microsoft/ai-103>